



Artificial Intelligence Foundations (CIS4049-N)

Problem-solving I: Uninformed Search – Part I

Module Leader: Dr Alessandro Di Stefano

Email: A.DiStefano@tees.ac.uk

School of **Computing, Engineering & Digital Technologies**

tees.ac.uk

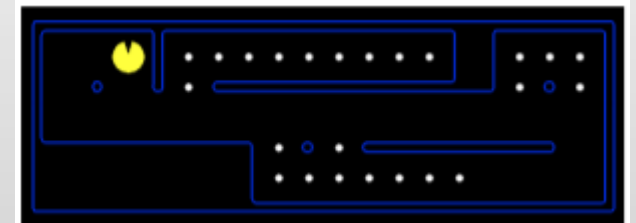
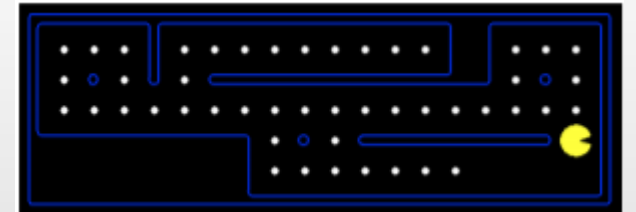
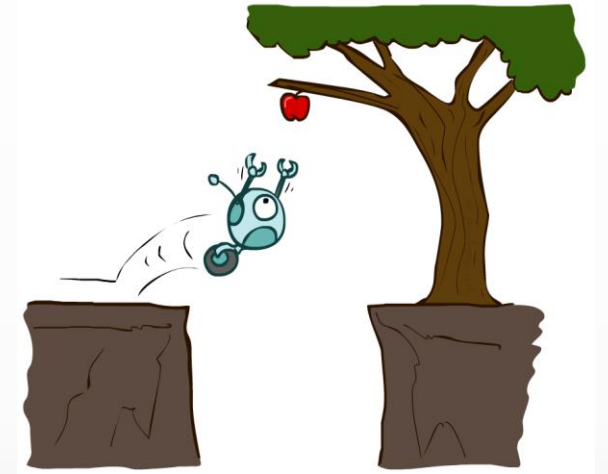
Today's Plan

- **Problem-Solving Agents**
- **Search problems (searching for solutions)**
- **Graphs and Networks – Basic Notions**
- **Uninformed Search algorithms:**
 - **Depth-First Search (DFS)**
 - **Breadth-First Search (BFS)**

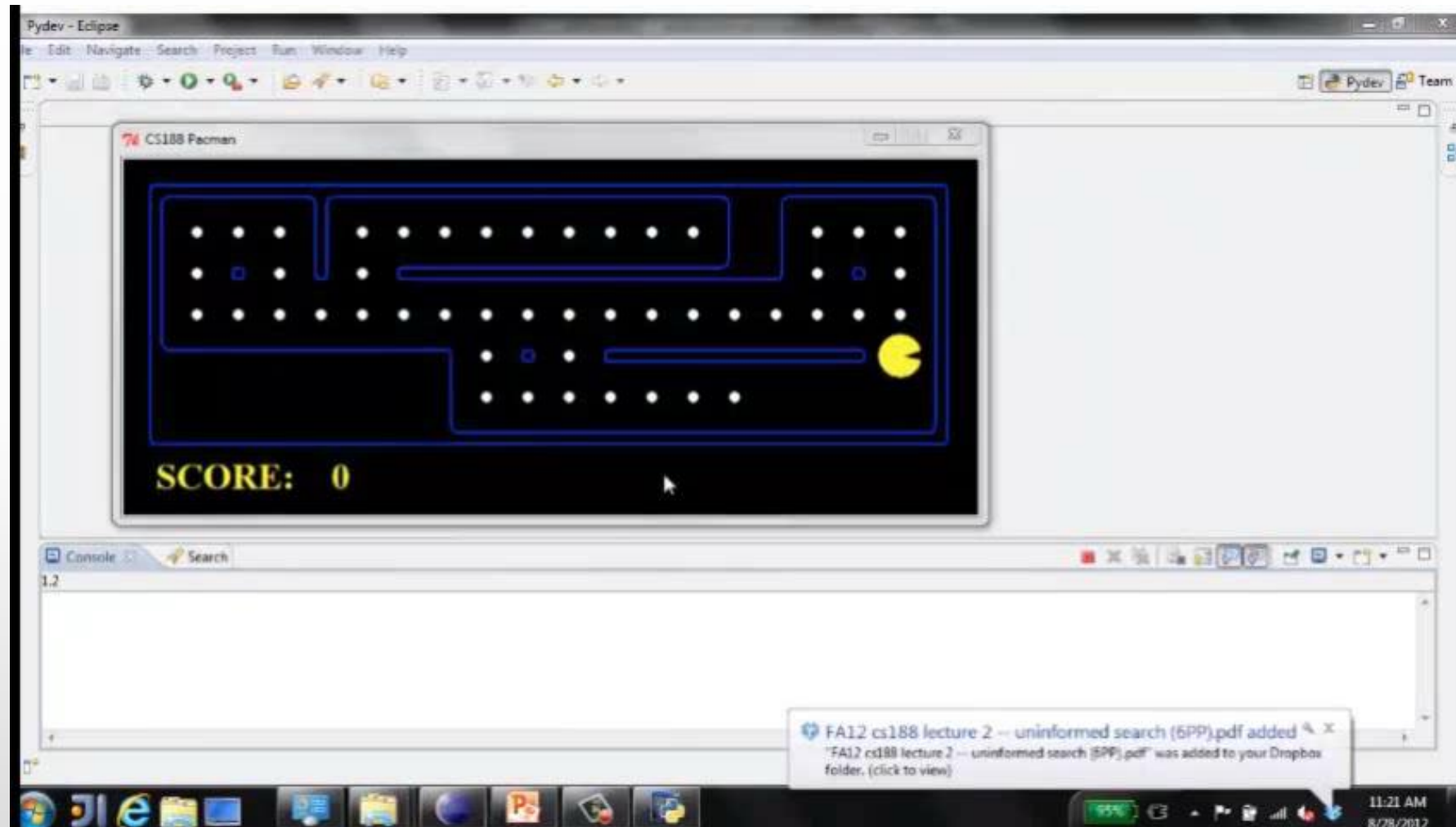
Problem-solving Agents

Reflex Agents

- Choose **action** based on **current percept** (and maybe memory)
- May have **memory** or a **model** of the world's **current state**
- **Do not consider** the **future consequences** of their **actions**
- Consider how the world IS.
- Can a reflex agent be *rational*?
- Examples:
 - Stove: hot, ouch!
 - Car: deer, brake!
 - Go to Tahoe: skid in snow, ???



Video of Demo Reflex Odd



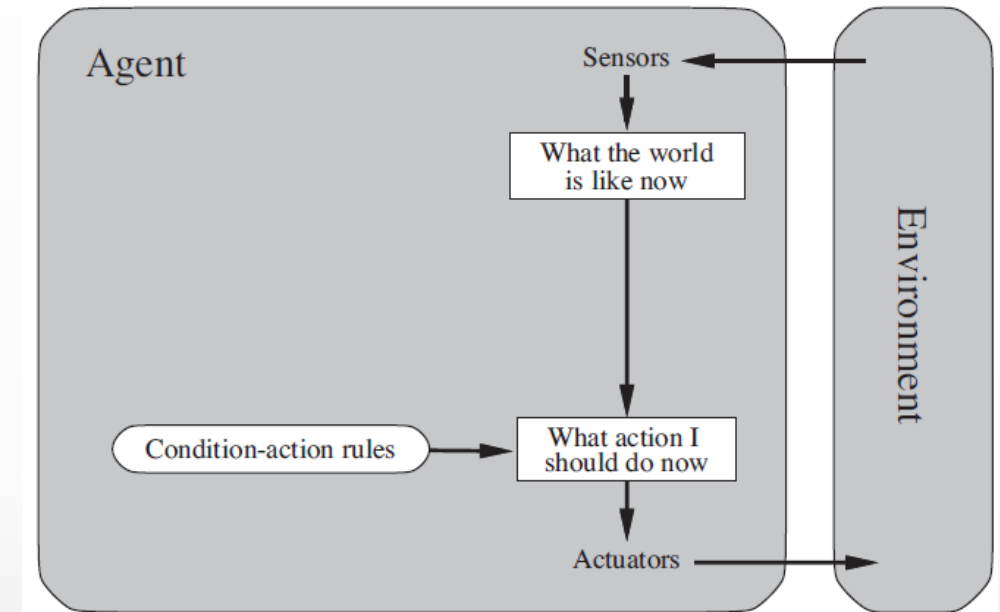
<https://inst.eecs.berkeley.edu/>

Types of AI Agents

- Simple Reflex Agents
- Model-based reflex agents
- Goal-based agents
- Utility-based agents
- Learning agents

Simple Reflex agents

- Select actions on the basis of the current percept, ignoring the rest of the percept history
- **Condition-action rule**
- **Limitations**
 - Simple, but **limited intelligence**
 - **No knowledge** about the non-perceptual parts of the system
 - Operating in a partially observable environment, **infinite loops** are **unavoidable**
 - Possible solution: **randomization**



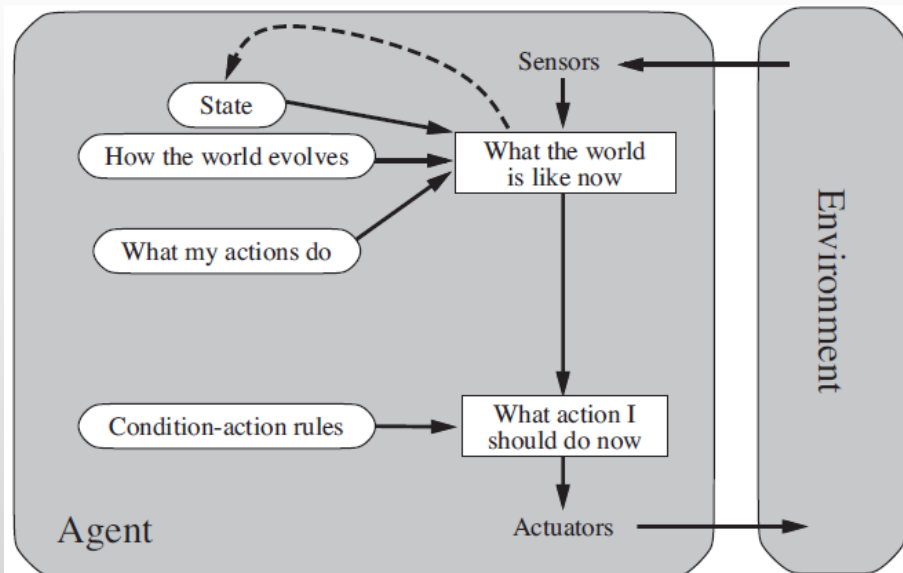
```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition–action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Russell, S.J. and Norvig, P., (2019). Artificial intelligence: a modern approach.

Model-based Reflex agents

- *Keep track of the part of the world it can't see now* – **internal state**
- **Model** of the world:
 - Information about how the world evolves independently of the agent
 - Information about how the agent's own actions affect the world



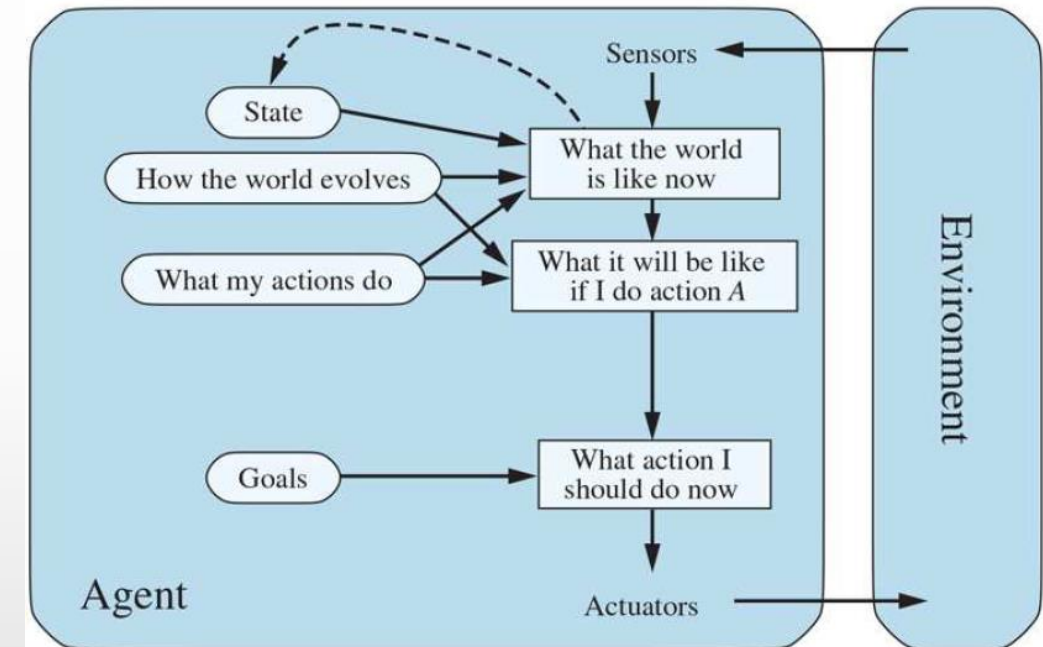
```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
               model, a description of how the next state depends on current state and action
               rules, a set of condition-action rules
               action, the most recent action, initially none

  state ← UPDATE-STATE(state, action, percept, model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Russell, S.J. and Norvig, P., (2019). Artificial intelligence: a modern approach

Goal-based agents

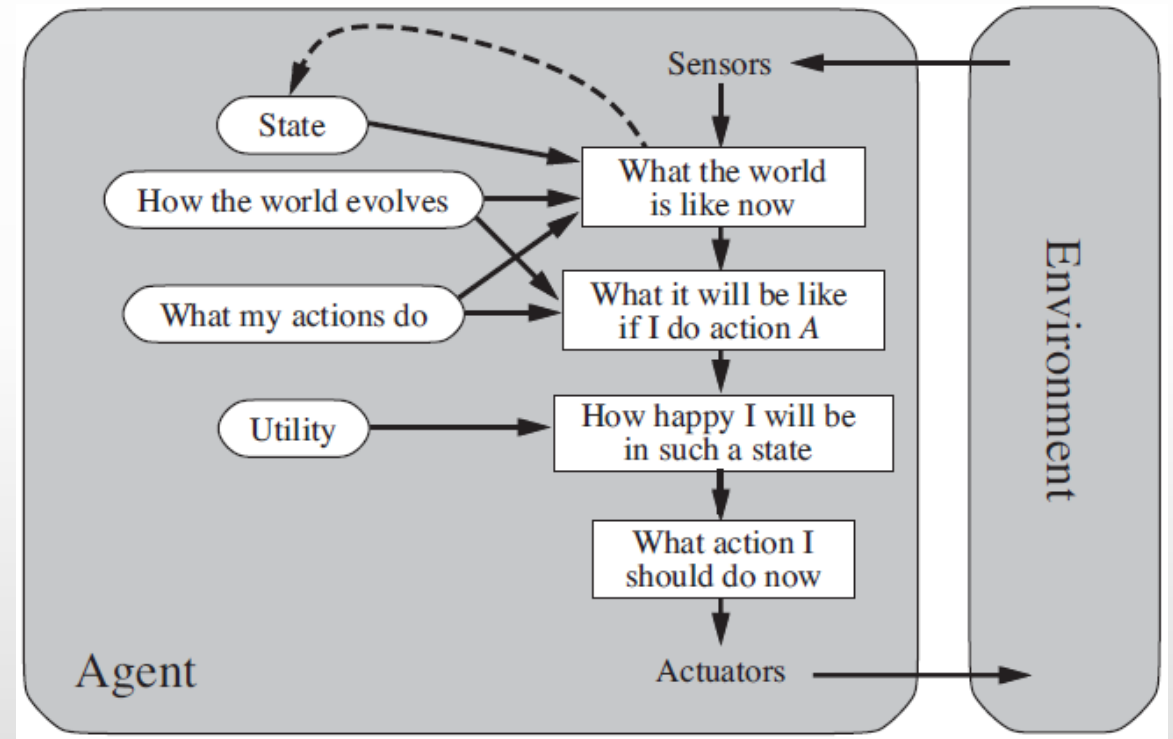
- The knowledge of the current state environment is **not always sufficient** to decide for an agent to what to do
- The agent needs to know its **goal** which describes desirable situations
- **Goal-based agents** expand the capabilities of the model-based agent by having the “**goal**” information
- They choose an **action**, so that they can achieve the **goal**
- These agents may have to consider a long sequence of possible actions before deciding whether the goal is achieved or not
- Such considerations of different scenario are called **searching** and **planning**, which make an agent **proactive**



Russell, S.J. and Norvig, P., (2019). Artificial intelligence: a modern approach.

Utility-based agents

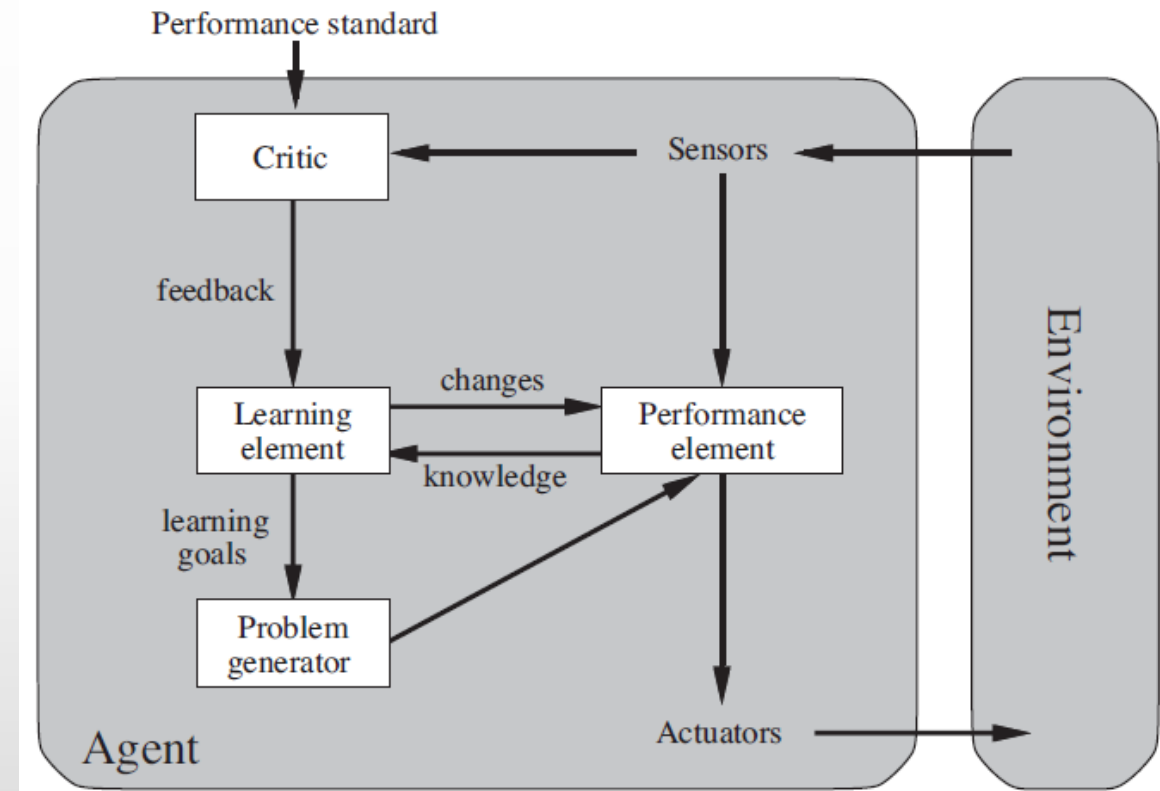
- Similar to the goal-based agents but provide an extra component of **utility measurement** which makes them different by providing a measure of success at a given state
- **Utility-based agent** act based not only goals but also the best way to achieve the goal
- The utility-based agent is useful when there are **multiple possible alternatives**, and an agent must choose in order to perform the **best action**
- **The utility function** maps each **state** to a **real number** to check how **efficiently** each action achieves the goals



Russell, S.J. and Norvig, P., (2019). Artificial intelligence: a modern approach.

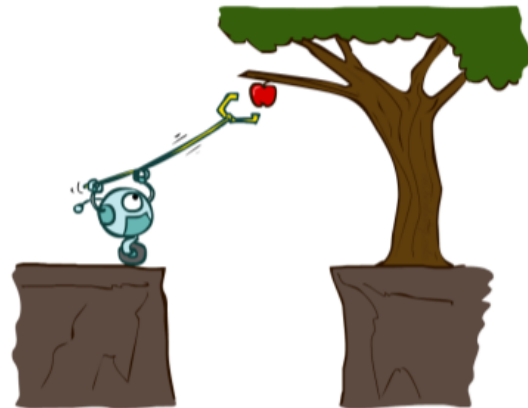
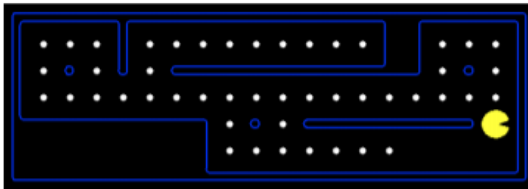
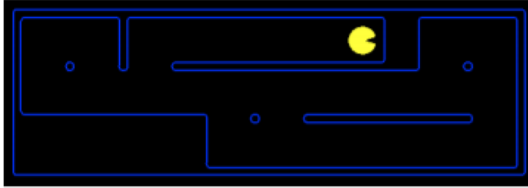
Learning agents

- A learning agent can **learn** from its **past experiences** (learning capabilities)
- It starts to act with basic knowledge and then able to **act** and **adapt automatically** through learning
- A learning agent has mainly **four conceptual components**:
 - **Learning element**: responsible for making improvements by learning from environment
 - **Critic**: Learning element takes feedback from critic which describes how well the agent is doing with respect to a fixed performance standard
 - **Performance element**: responsible for selecting external action
 - **Problem generator**: responsible for suggesting actions that will lead to new and informative experiences
- Hence, learning agents are able to **learn, analyse performance**, and look for **new ways to improve the performance**



Russell, S.J. and Norvig, P., (2019). Artificial intelligence: a modern approach.

Planning agents



- Ask “**what if?**”
- Decisions based on **(hypothesized) consequences of actions**
- Requirement – Agent must have a **model** of how the world evolves in response to actions
- Must formulate a **goal** (or test) – goal (or condition) achieved or not?
- Consider how the world **WOULD BE**
 - Optimal or not optimal – **Optimal**: achieve goal with the minimum cost
 - Complete or not – **Complete**: find the solutions (if exists)
 - Planning vs. replanning

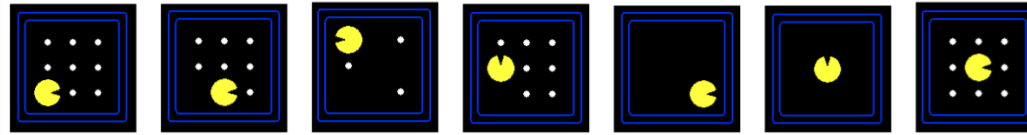
Search problems

Search Problems



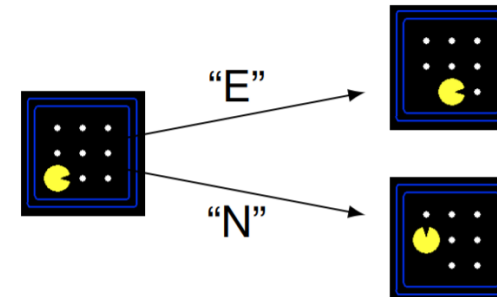
- A **search problem** consists of:

- A **state space**



- A **successor function** (with actions, costs)

For any given state, available actions and consequences



- A **start state** – where the agent is, and a **goal test** – condition to be met

- A **solution** is a sequence of actions (a **plan**) which transforms the **start state** to a **goal state**

Search Problems - Models

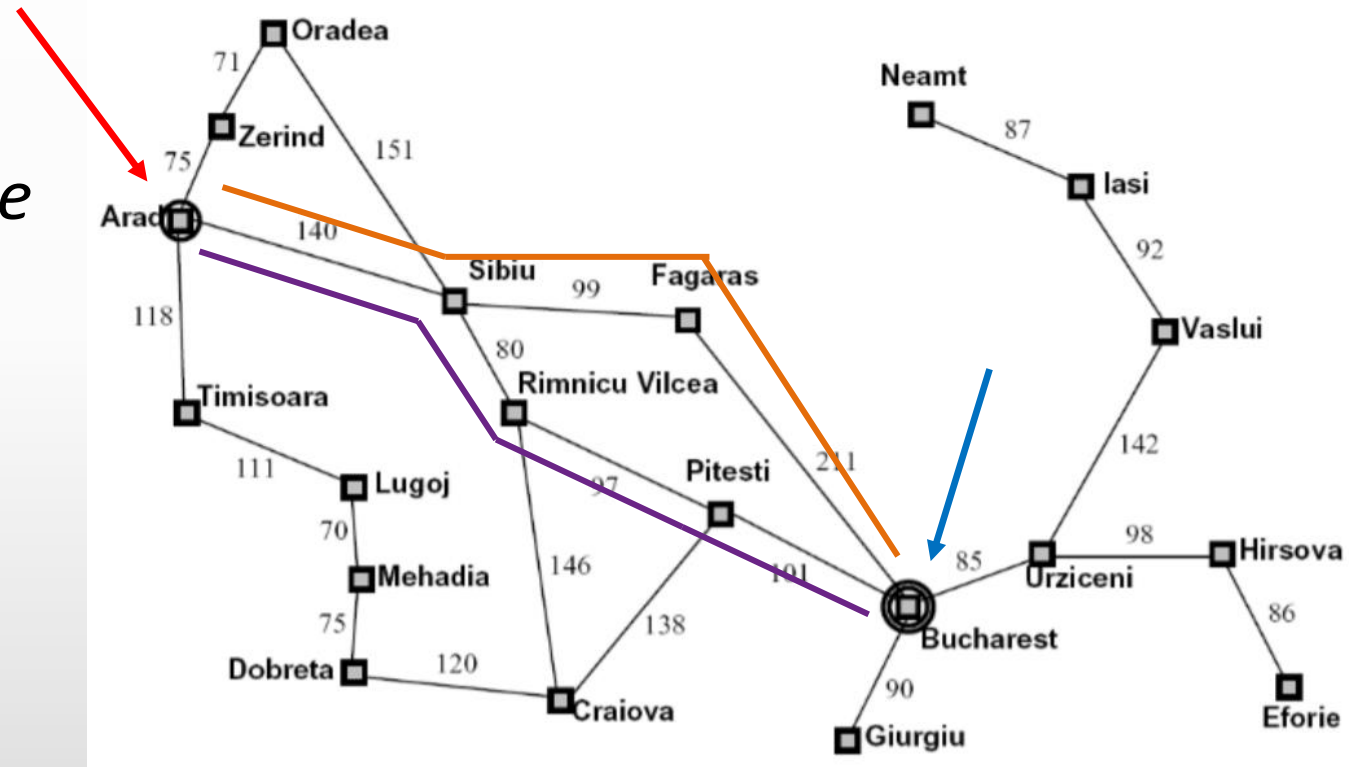
What is a **good model** of the **problem** that you're trying to solve in the **real world**?



<https://inst.eecs.berkeley.edu/>

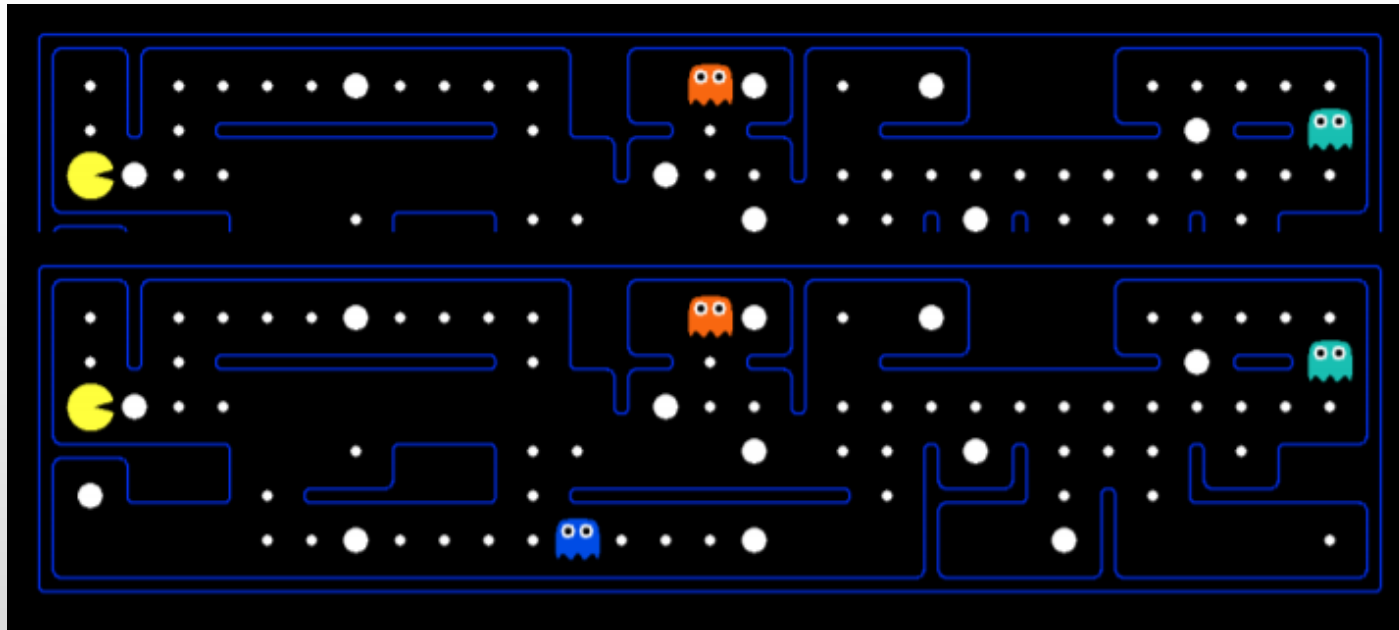
Example – Travelling in Romania

- **State space:** Cities
- **Successor function:** Roads -
Neighboring city with *cost = distance*
- **Start state:** Arad
- **Goal test:** Is state == Bucharest?
- **Solution?**

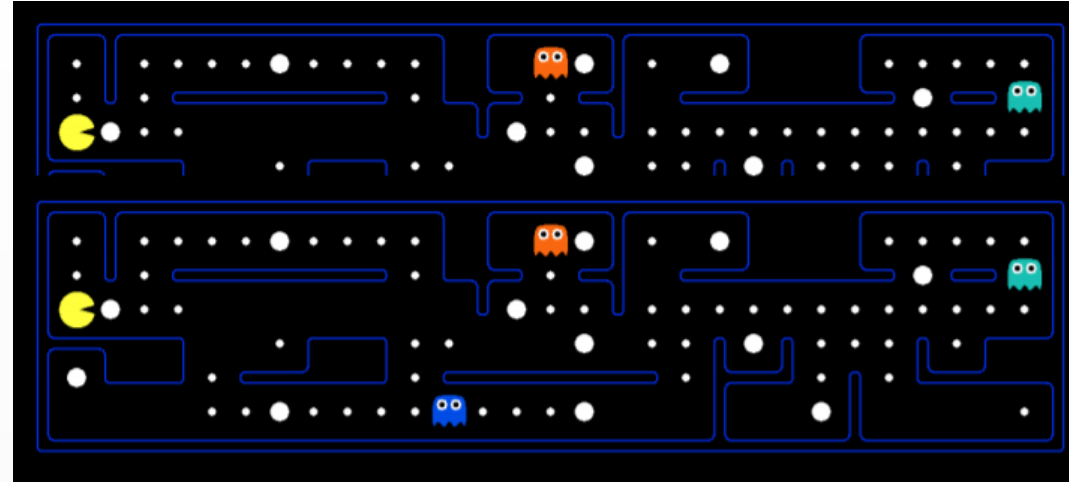


What's in a State Space

- The **world state** includes every last detail of the environment
- **Search state** keeps only details needed for planning (**abstraction**)



What's in a State Space



- **Problem:** Pathing

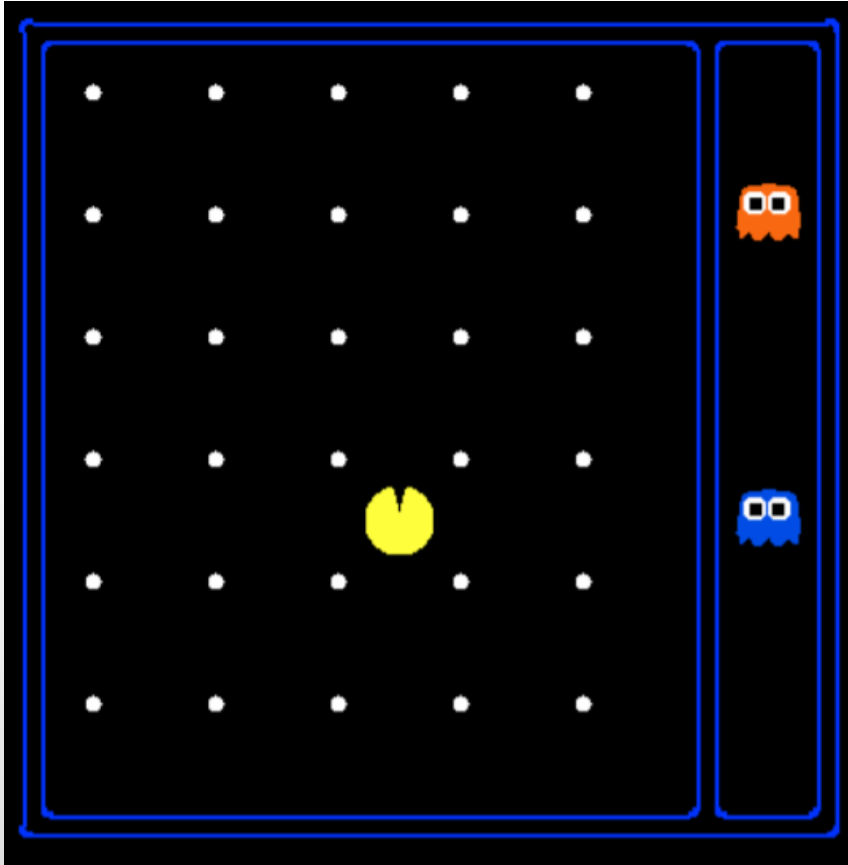
- **Pathing States:** (x,y) location
- **Actions:** NSEW
- **Successor:** update location only
- **Goal test:** is (x,y) == END?



- **Problem:** Eat-All-Dots

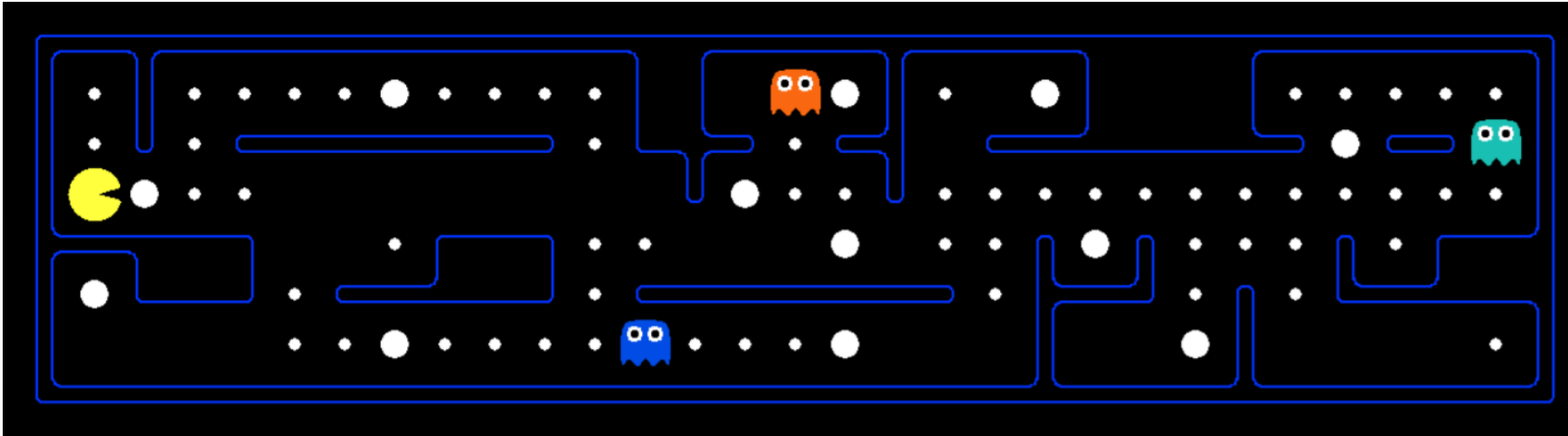
- **States:** {(x,y), dot booleans}
- **Actions:** NSEW
- **Successor:** update location and possibly a dot Boolean
- **Goal test:** dots all false

State Space Sizes



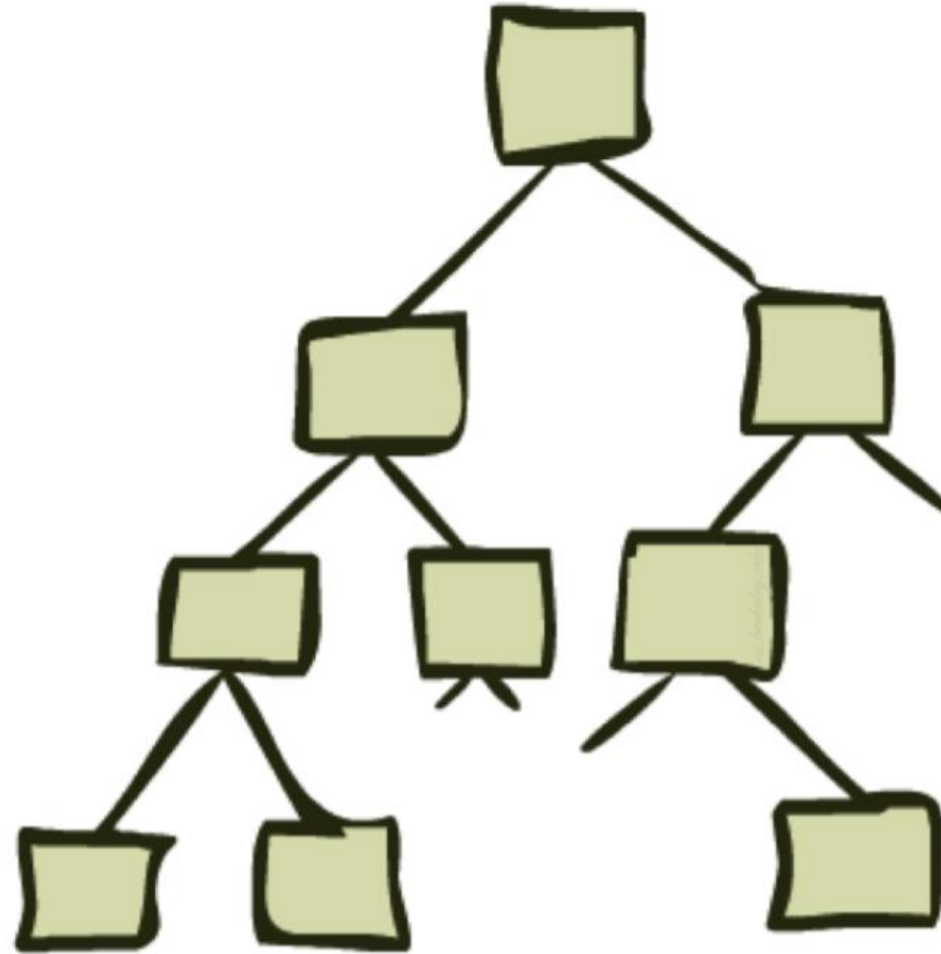
- **World state**
 - Agent positions: 120
 - Food count: 30
 - Ghost positions: 12
 - Agent facing: NSEW
- How many World states? $120 \times (2^{30}) \times (12^2) \times 4$
- States for pathing? **120**
- States for eat-all-dots? $120 \times (2^{30})$

Problem – Safe Passage



- **Problem:** eat all dots while keeping the ghosts perma-scared?
- **What does the state space have to specify?**
 - agent position (i.e., PacMan location)
 - dot Booleans (i.e., food location)
 - power pellet Booleans (i.e., power pellet location)
 - remaining scared time (i.e., timer for the ghosts)

State Space Graphs and Search Trees



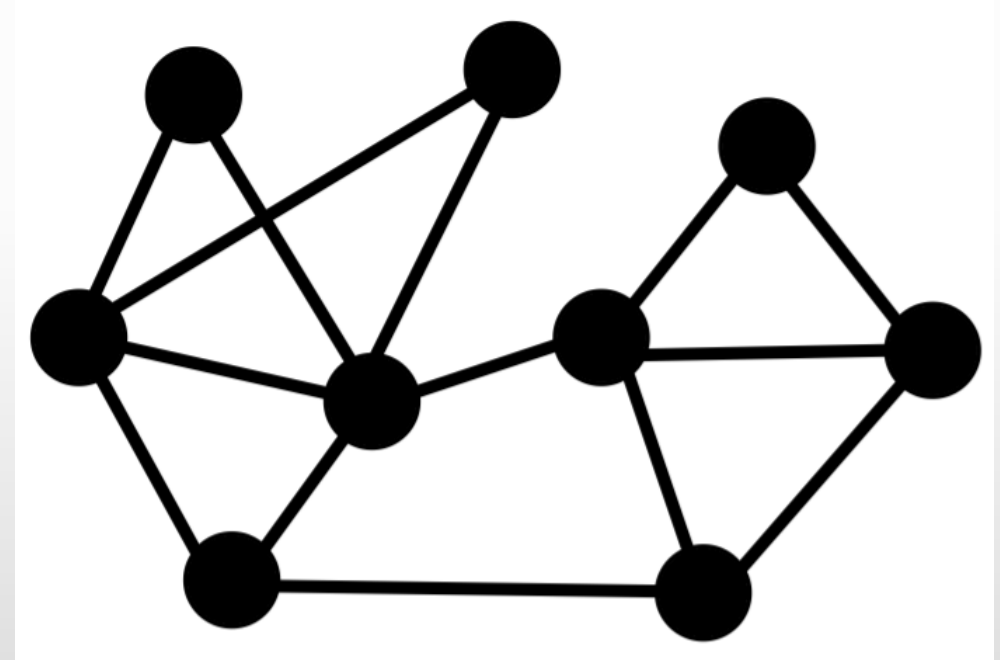
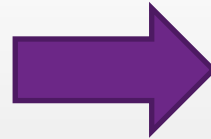
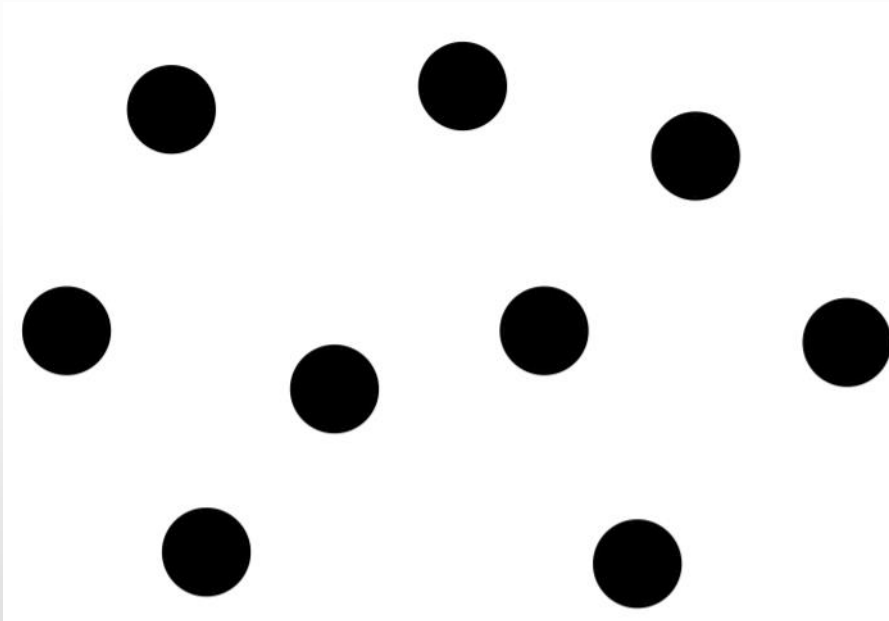
Graph and Networks - Basic notions

Graph and Networks - Definition

- A **Graph** is a mathematical object composed by **vertices** (or nodes) connected by **edges**
- We use the term **Network** for any realisation of a graph (often refers to a real-world system of interconnected components, emphasizing the practical application and functioning of the interconnected entities)
- In summary, a **graph** is an abstract mathematical structure, whereas a **network** can refer to both the abstract structure and its real-world instantiation in various fields

Why graphs?

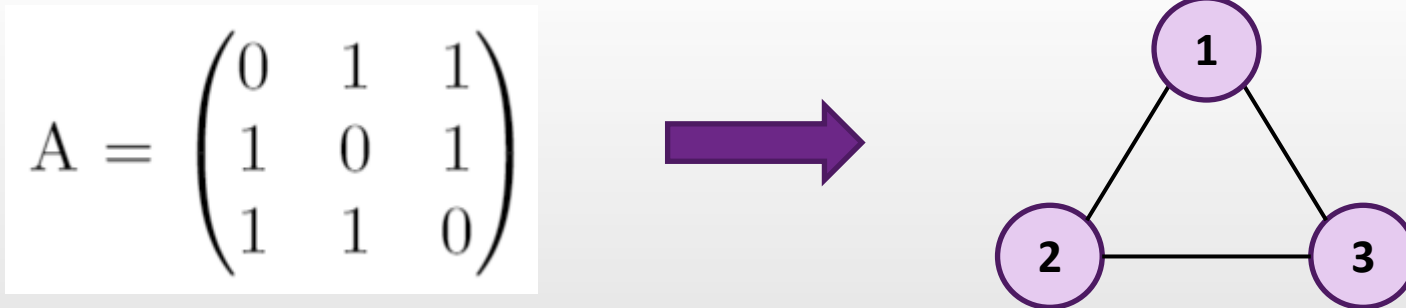
- **Graphs** are a general language for describing and analysing entities with relations/interactions



GRAPH

Graphs: Notations & Terminology

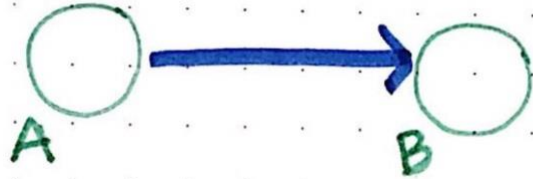
- A network can be represented by a **graph** (V, A) , which consists of a set of **nodes or vertices** $V = \{1, \dots, N\}$ and an $N \times N$ matrix $\mathbf{A} = [a_{ij}]$, $i, j \in V$.
- **Matrix A** is referred to as an **Adjacency matrix**, where $a_{ij} \in \{0, 1\}$ represents the presence of an edge connecting nodes i and j .



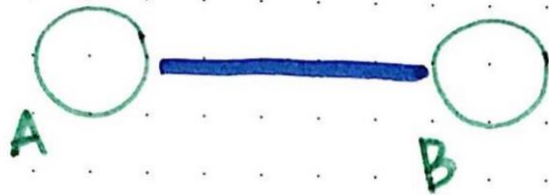
- Another representation of a graph is given by (V, E) , where V is the set of **vertices** and E is the set of **edges** in the network.
 - $E = \{(1, 2), (1, 3), (2, 3)\}$

Directed and Undirected Graphs

Different types of edges in graphs



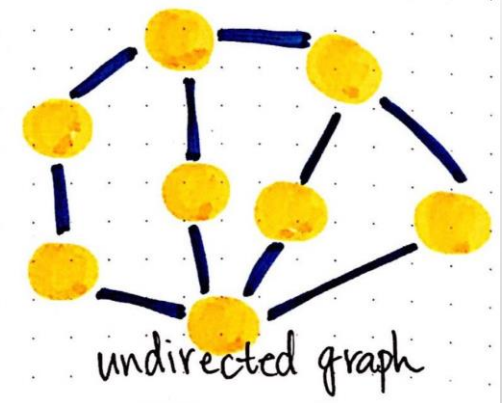
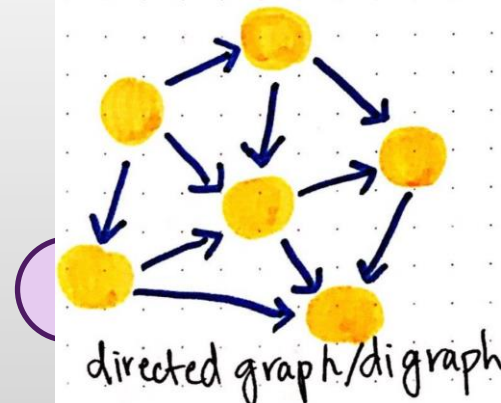
directed edge: there is only a path from A, the origin, to B, the destination



undirected edge: the path between A and B is bidirectional, meaning origin & destination are not fixed.

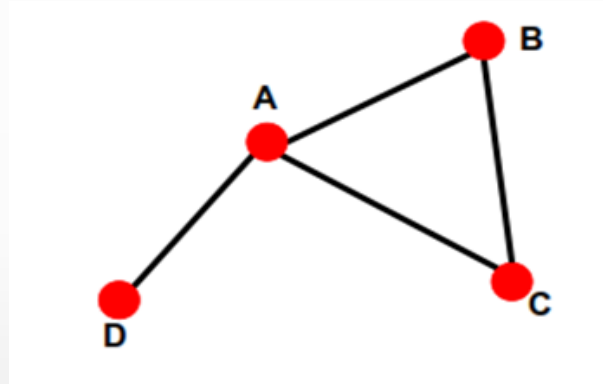
$a_{ij} \neq a_{ji}$ and an **undirected**

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

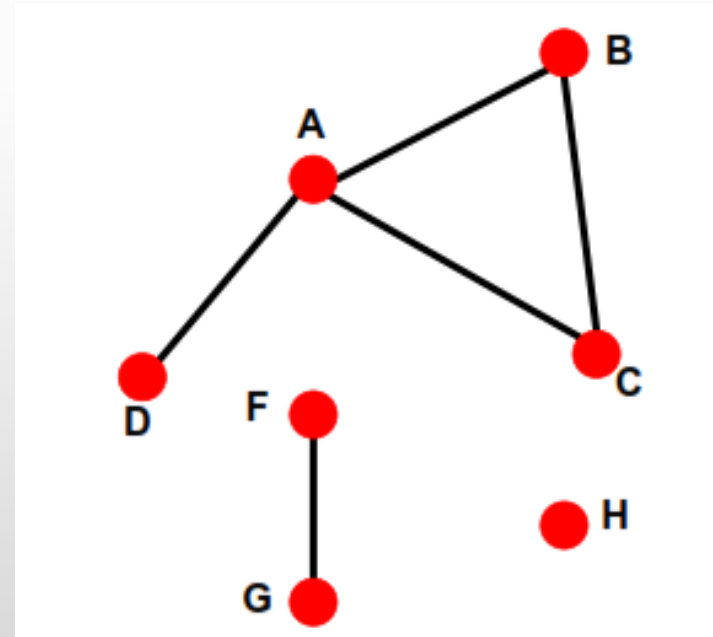


Connectivity of Graphs

- **Connected Graph**
 - Any node can reach any node via a path



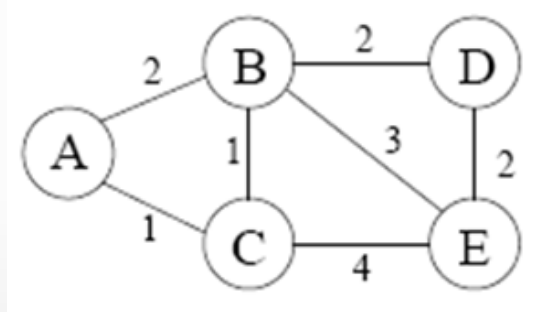
- A **disconnected graph** is made up by two or more connected components.



More types of Graphs - Weighted graphs/Complete graphs

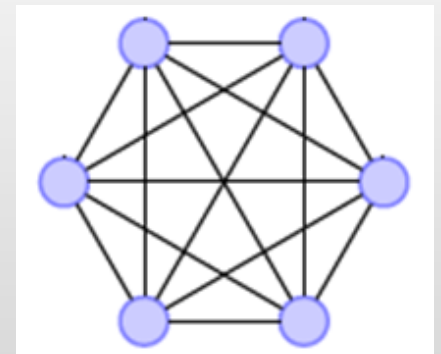
- **Weighted Graphs**

- The edge weight $g_{ij} > 0$ can also take on non-binary values, representing the intensity of the interaction, in which case we refer to (N, g) as a **weighted graph**.



- **Complete Graph**

- Graph with the **maximum number of edges** ($E = E_{max}$)
- For a graph with n nodes, that is $n(n - 1)/2$



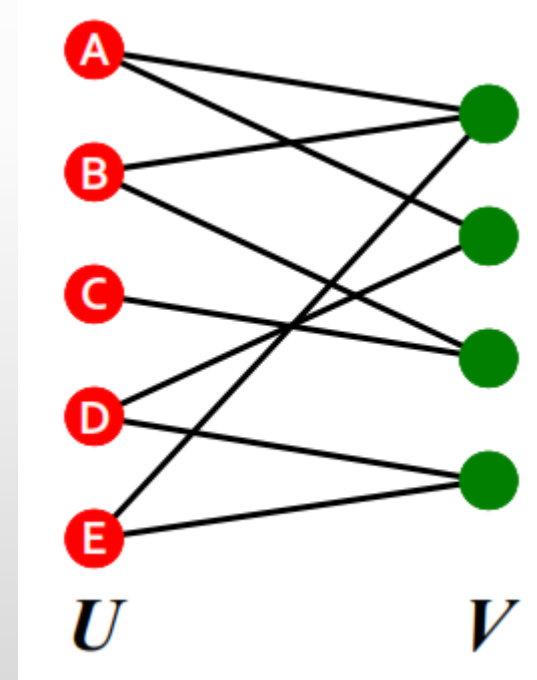
More types of Graphs: Bipartite Graphs

- **Bipartite graph**

- graph whose nodes can be divided into **two disjoint sets** U and V
- **every link connects a node in U to one node in V**
 - U and V are **independent sets**

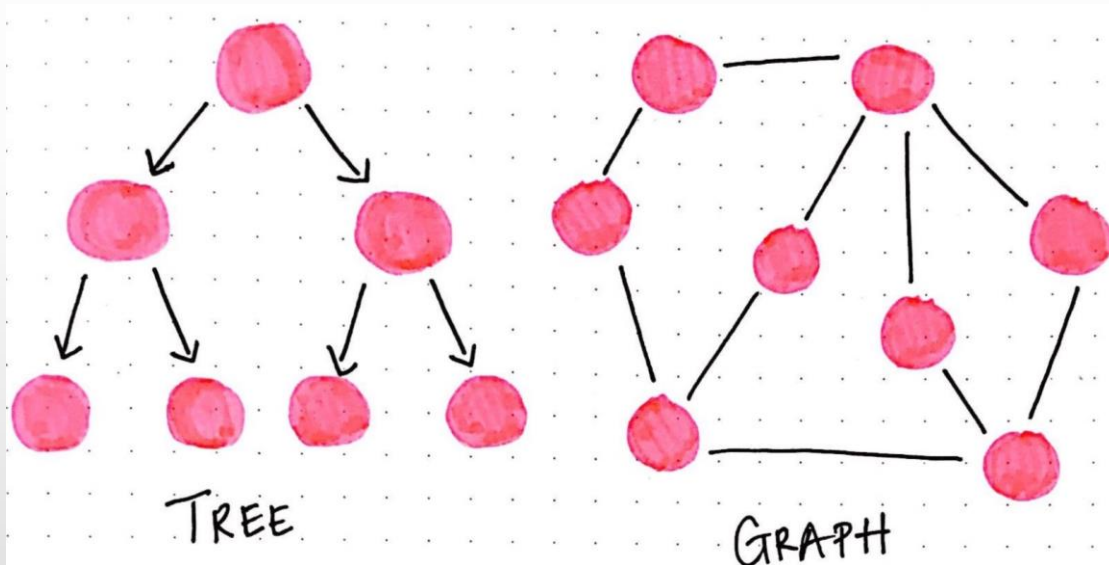
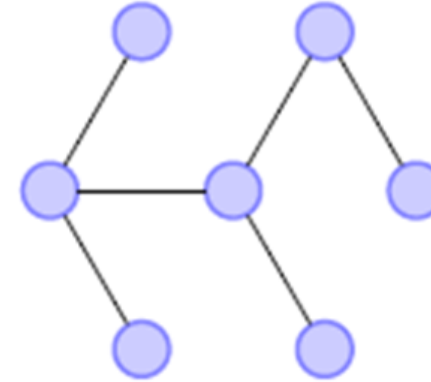
- **Examples**

- Authors to Papers
- Actors to Movies
- Customers to restaurants

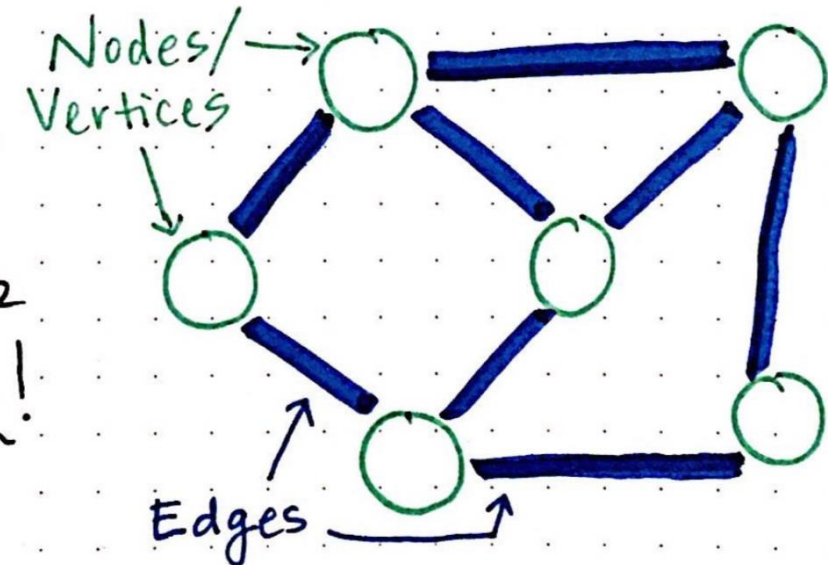


More types of Graphs: Trees

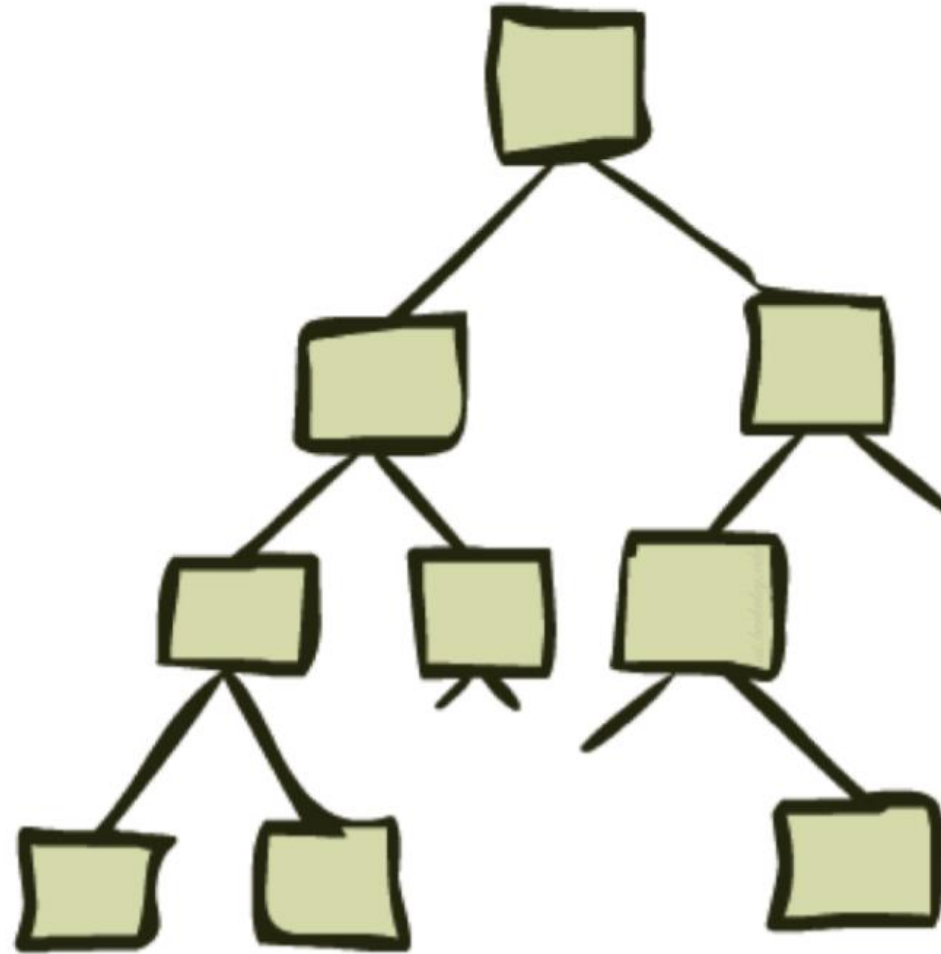
- A **tree** is a connected (undirected) graph with **no cycles**
 - A connected graph is a tree if and only if it has $n - 1$ edges
 - In a tree, there is a unique path between any two nodes



Edges can connect nodes in any possible way! No rules!

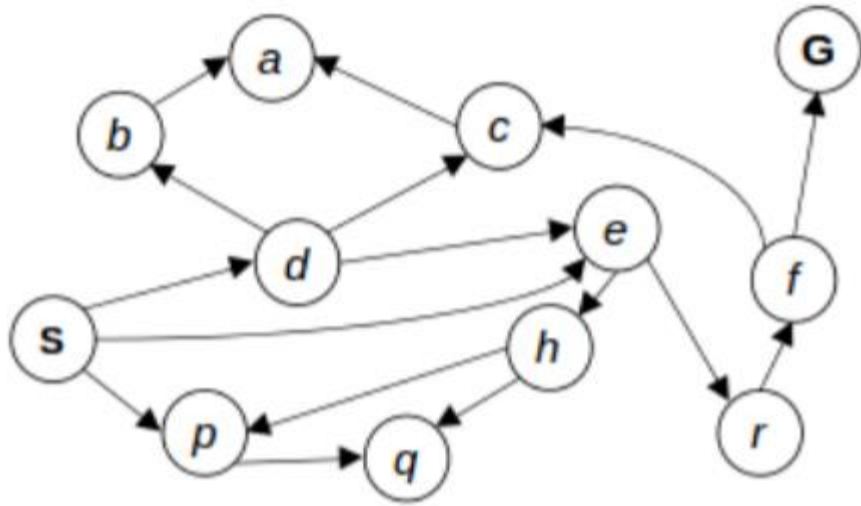


State Space Graphs and Search Trees



State Space Graphs

- **State space graph:** A mathematical representation of a **search problem**



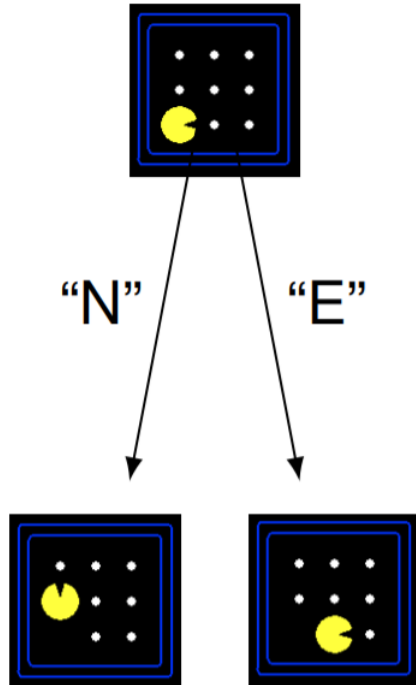
*Tiny search graph for a
tiny search problem*

- **Nodes** are (abstracted) **world configurations**
- **Arcs** represent **successors** (action results)
- The **goal test** is a set of **goal nodes** (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea (e.g., re-planning)

Search Tree

- **Search tree:** A “what if” tree of plans and their outcomes

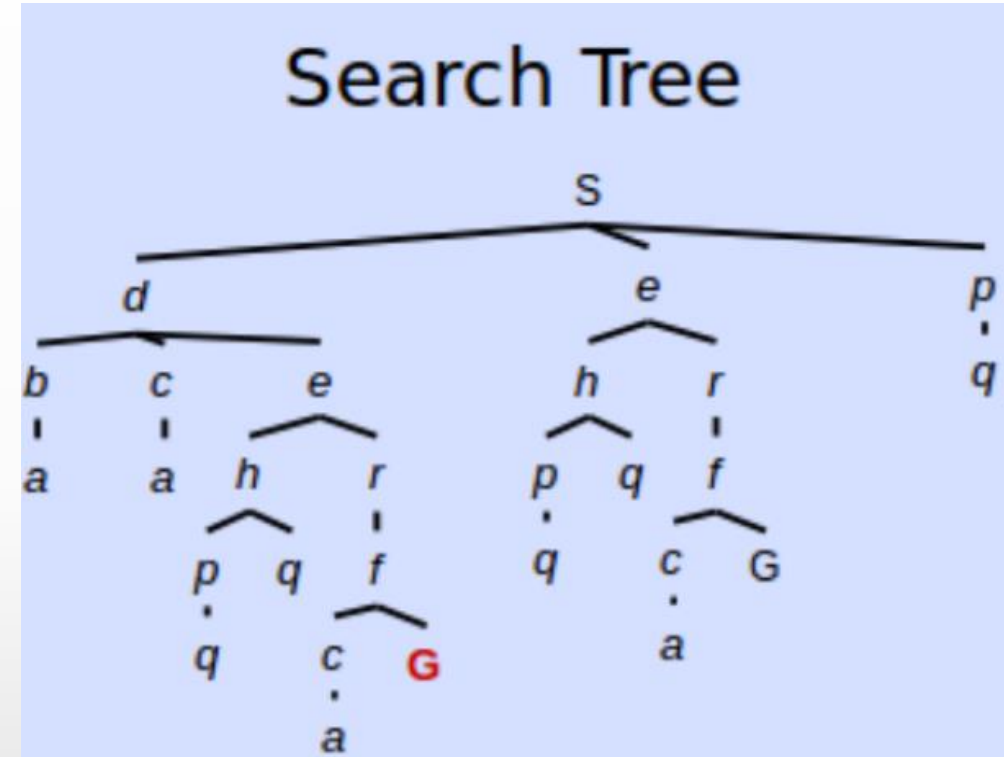
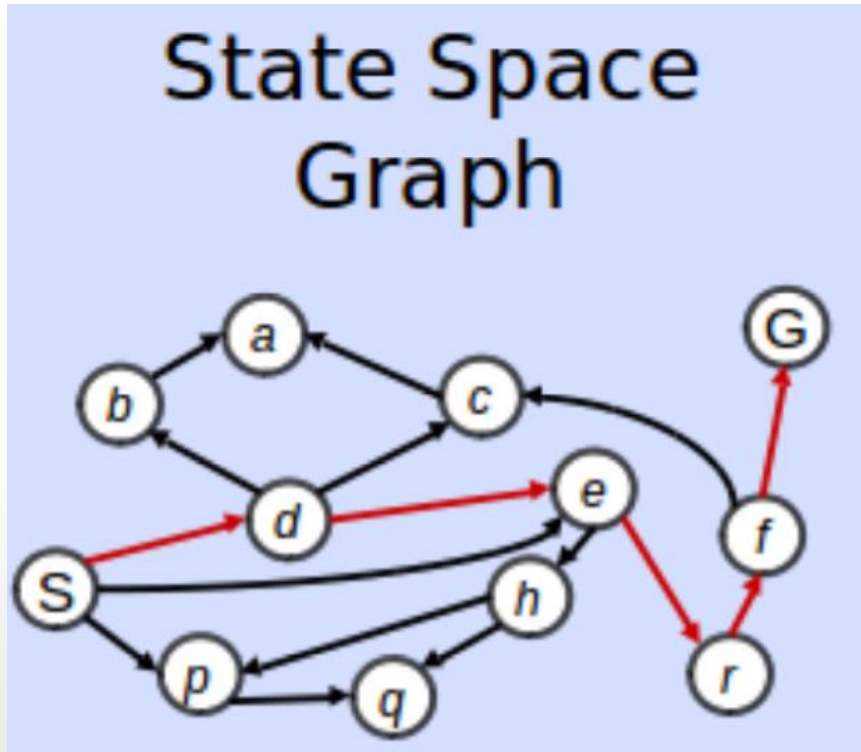
This is now / start



Possible futures

- The **start state**: *root* node
- **Children** correspond to **successors**
- Nodes show **states**, but correspond to **PLANS** (or sequences of actions) that achieve those states
- For most problems, we can never actually build the whole tree!! (even larger than the state space graph – multiple ways to get to the same state, hence multiple occurrences of the same state)

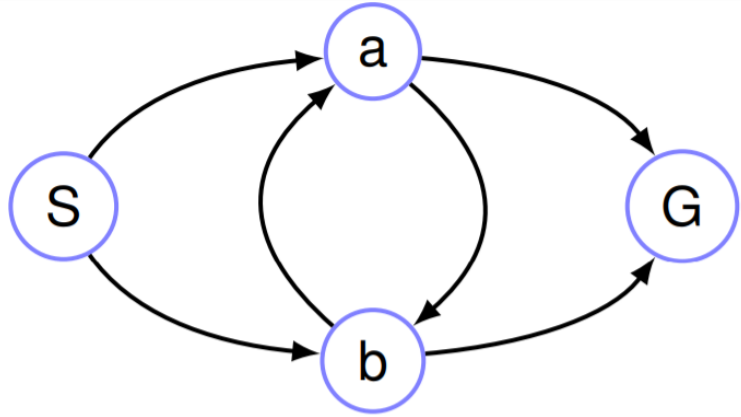
State Space Graphs vs Search Trees



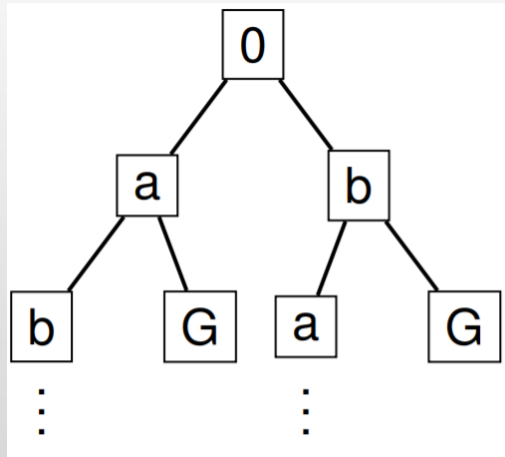
- Each NODE in search tree is an entire PATH in state space graph
- We construct both on demand – and we construct as little as possible

State Space Graphs vs Search Trees - Examples

- Consider this 4-state graph:

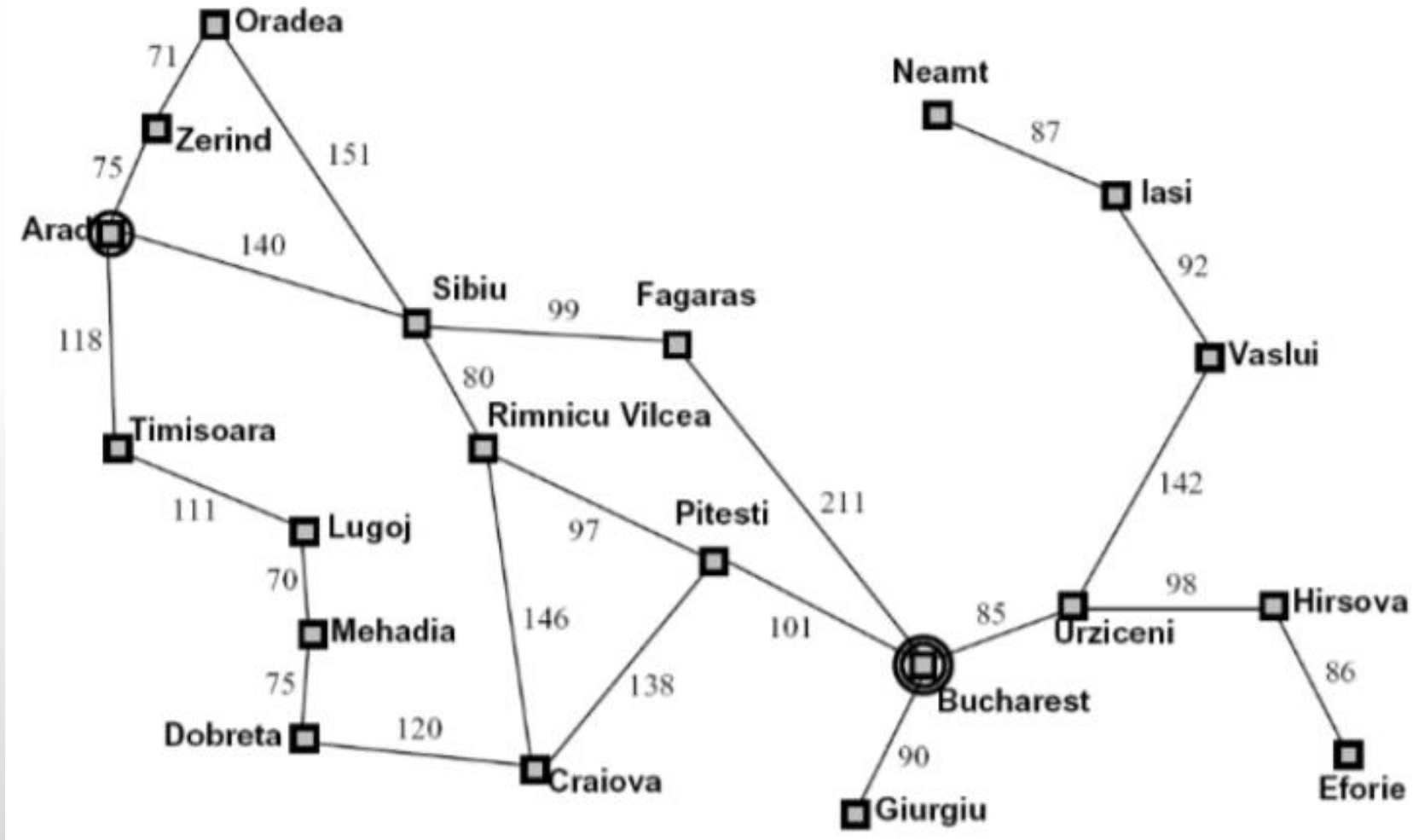


How large could the state space graph be? ∞



How large could search tree be? ∞

Search Example – Romania

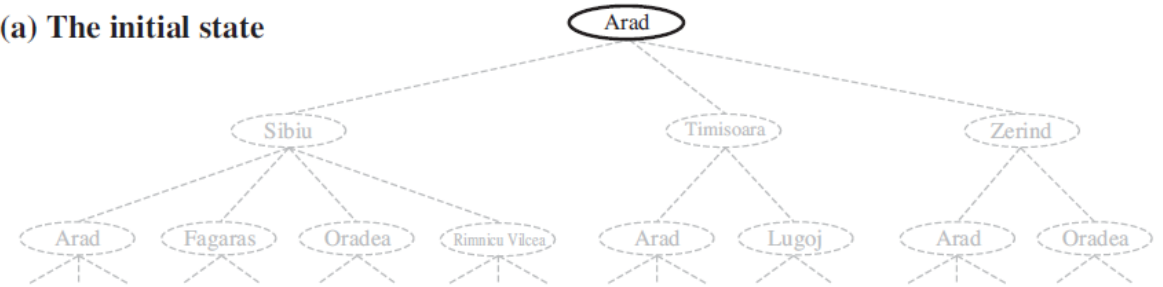


Searching with a Search Tree

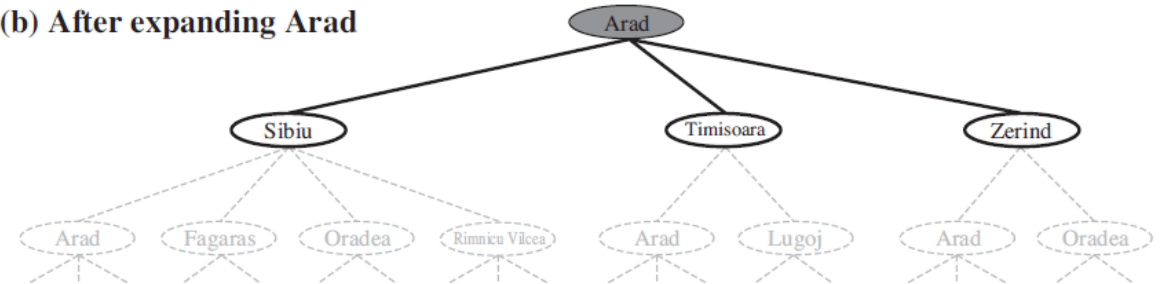
- **Search:**

- (1) Expand out **potential plans** (tree nodes)
- (2) Maintain a **fringe** of partial plans under consideration.
- (3) Try to **expand** as few tree nodes as possible

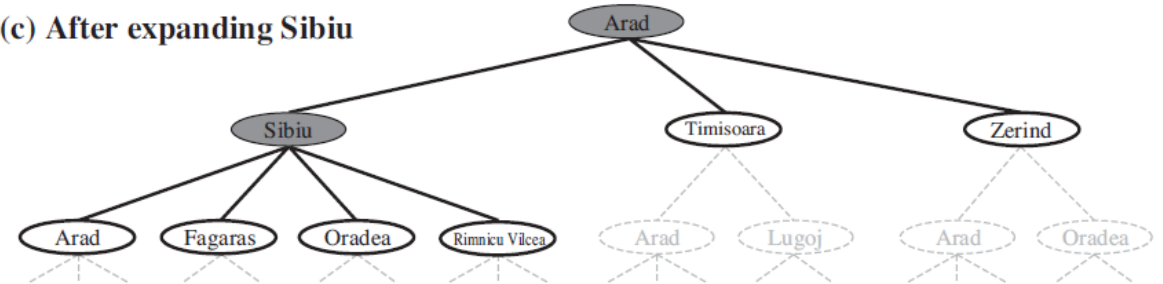
(a) The initial state



(b) After expanding Arad



(c) After expanding Sibiu

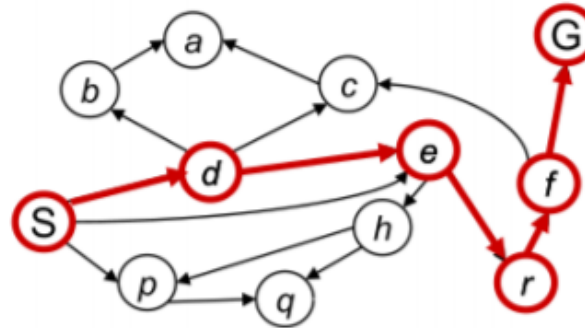


General Tree Search

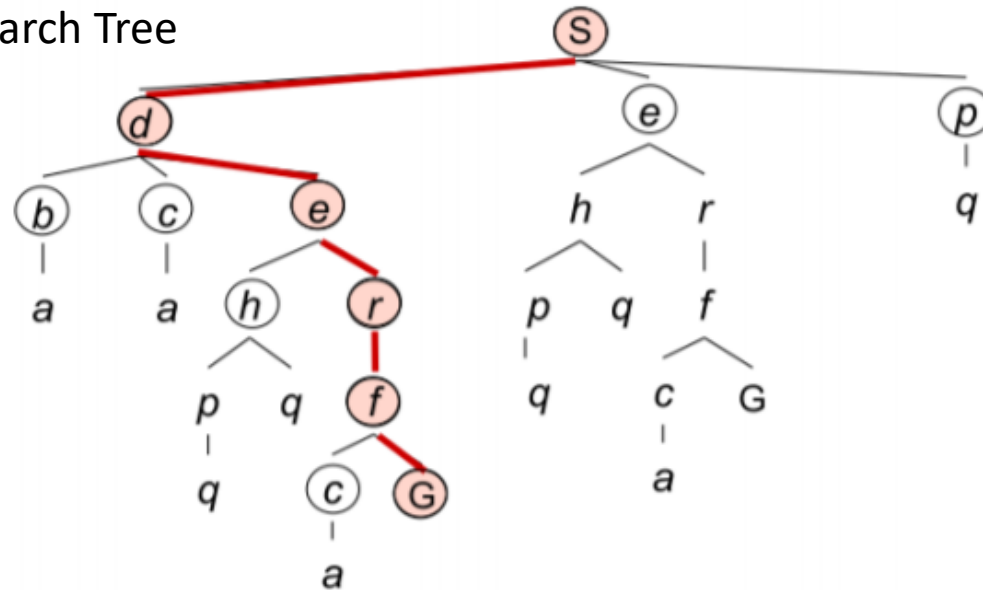
- Important **ideas**:
 - **Fringe** (set of leaf nodes waiting to be expanded)
 - **Expansion** (frontier or candidates for expansion - where you pick one out of the fringe and expand it, but before you do, of course, check if the goal might already be achieved)
 - **Exploration strategy**
- **Main question**: which fringe nodes to explore?

```
function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier
```

Tree Search - Example



Search Tree



~~s~~

~~s → d~~

s → e

s → p

s → d → b

s → d → c

~~s → d → e~~

s → d → e → h

~~s → d → e → r~~

~~s → d → e → r → f~~

s → d → e → r → f → c

~~s → d → e → r → f → G~~

Fringe in code

Uninformed Search Methods

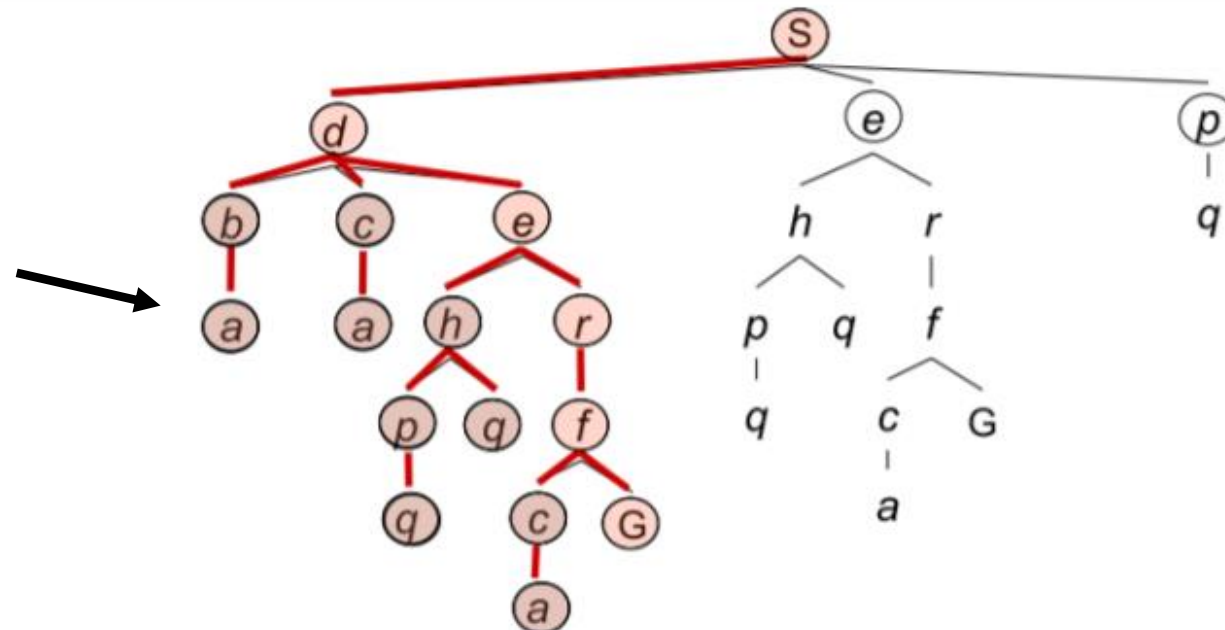
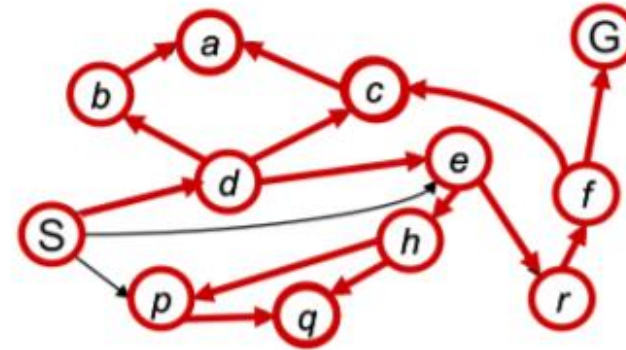
Depth-First Search (DFS)



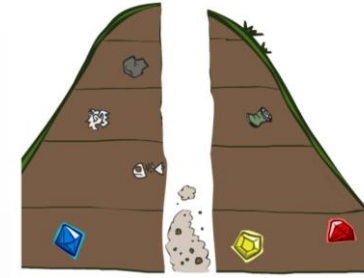
Tree Search - Example

*Strategy: expand a
deepest node first*

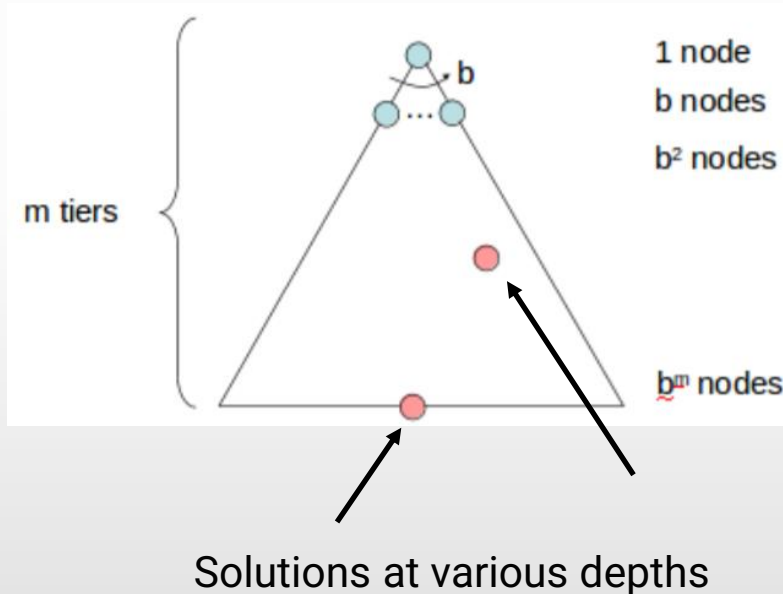
*Implementation:
Fringe is a LIFO stack*



Search Algorithms – Properties



Search tree cartoon

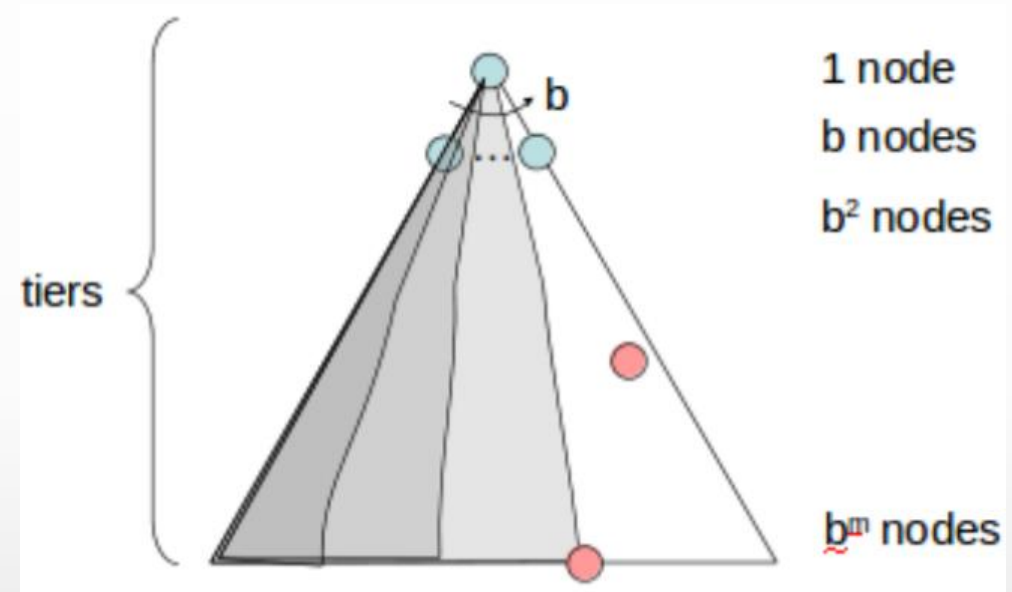


- **Complete:** Guaranteed to find a solution if one exists?
- **Optimal:** Guaranteed to find the least cost path?
- **Time complexity?** - how long does the compute take to find a solution?
- **Space complexity?** - how much memory do you need in the process of that compute to get the computation done?
- Sketch of search tree:
 - **b** is the **branching factor** (how many successors are there from any given node?)
 - **m** is the **maximum depth**
 - **solutions** at various depths
- Number of nodes in entire tree?

$$1 + b + b^2 + \dots = O(b^m)$$

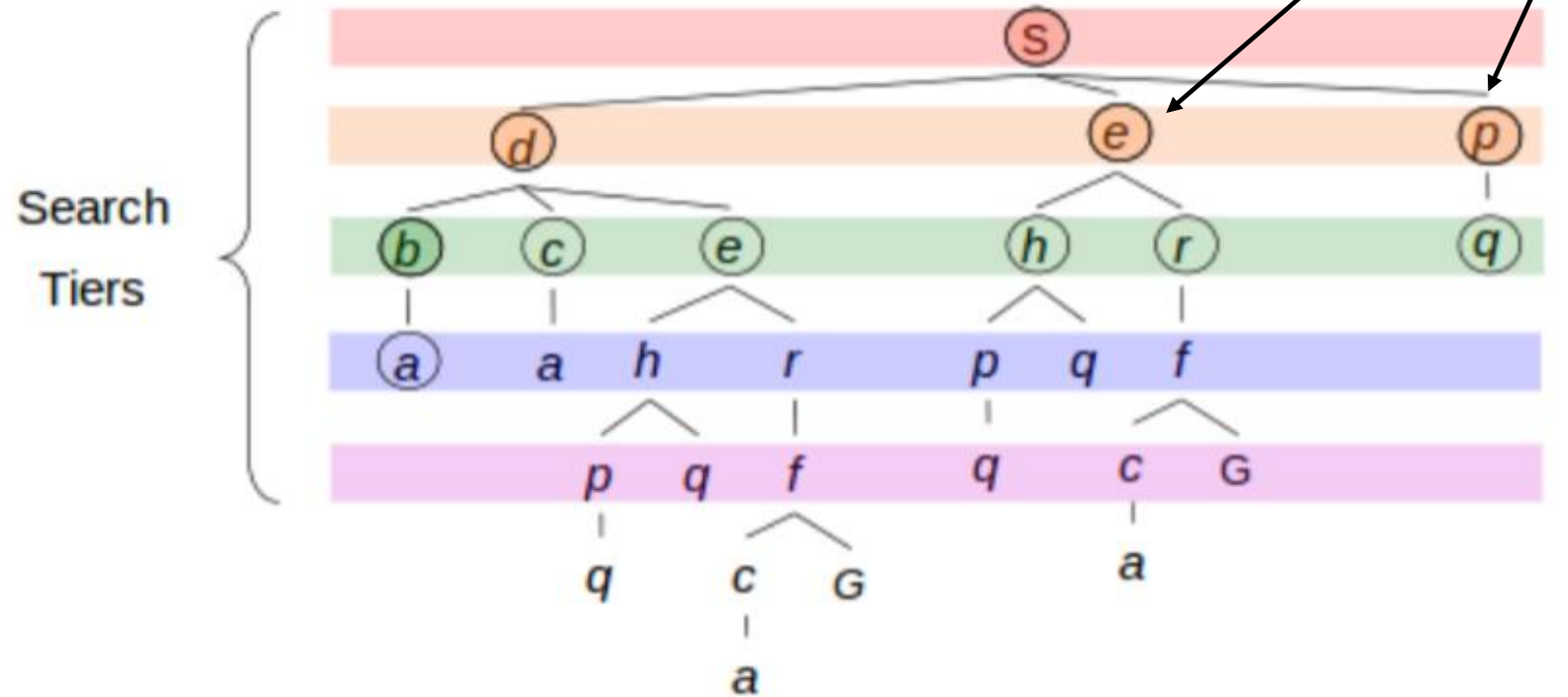
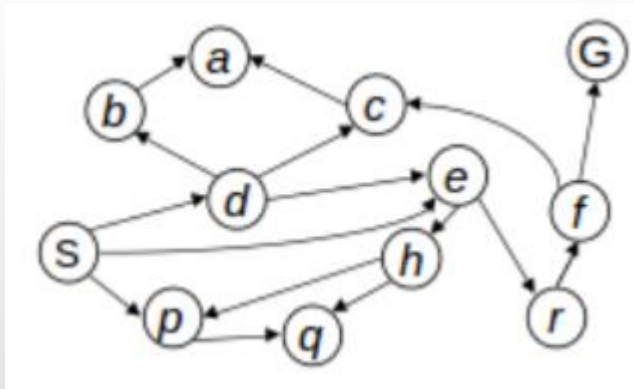
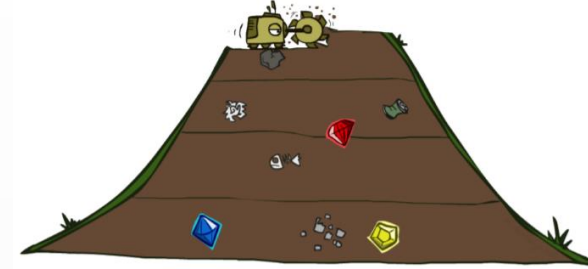
Depth-First Search – Properties

- Which nodes expanded?
 - Some left prefix of the tree
 - Could process the whole tree!
 - If m is finite, takes time $O(b^m)$
- How much space does the fringe take?
 - Only has siblings on path to root, so $O(b^m)$
- Is it complete?
 - m could be infinite, so only if we prevent cycles (more later)
- Is it optimal?
 - No, it finds the “leftmost” solution, regardless of depth or cost



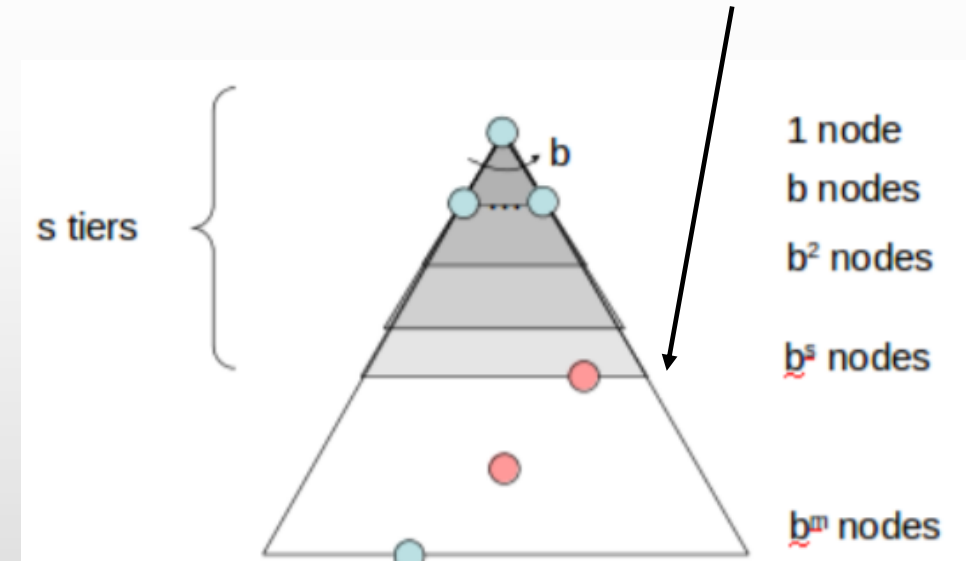
Breadth-First Search (BFS)

- **Strategy:** expand a shallowest node first
- **Implementation:** Fringe is a FIFO stack



Breadth-First Search - Properties

- What nodes does BFS expand?
 - Processes all nodes above shallowest solution
 - Let depth of shallowest solution be $s \rightarrow$ Search takes time $O(b^s)$
- How much space does the fringe take?
 - Has roughly the last tier, so $O(b^s)$
- Is it complete?
 - s must be finite if a solution exists, so yes!
- Is it optimal?
 - Only if costs are all 1 (more on costs later)

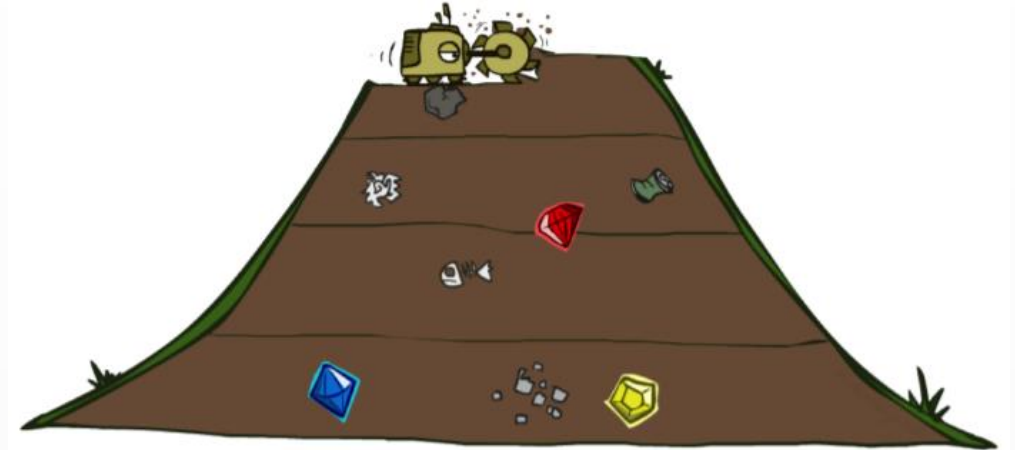


DFS vs BFS



DFS

VS



BFS

- When will BFS outperform DFS? - If the goal is shallow and to the right in the tree
- When will DFS outperform BFS? - When there are many deep goals in the tree, so you need to go deep anyway (memory limitations – DFS outperforms BFS)
- **Space** versus **Time** or **Quality** of Solution (trade-off)

What you just learned and what's next?

- Problem-Solving Agents – **Planning Agents**
- **Search problems - Searching for Solutions**
- **Uninformed Search Methods**
- Next week – **Other uninformed Search Methods**
 - Iterative Deepening Depth-First Search (IDDFS)
 - Cost-Sensitive Search
 - Uniform Cost Search (UCS)

Questions?

A.DiStefano@tees.ac.uk